empeo Registration Automation

Submission document: demo, test cases, how to use, conclusion and source code · Gofive Automation Tester Exam 1.0

System under test	https://portal.uat.gofive.co.th/Register/empeo (UAT)
Stack	Playwright Test 1.56.1 · TypeScript 5.6 · Node.js 22 · Chromium
Pattern	Page Object Model · custom fixtures · data-testid locators · text-as-constants
Repository	github.com/stormnoopper/gofive-empeo-registration-automation (private, access on request)

42	42 / 42	~1.3 min	4
automated test runs	passed, no retries	full suite, 4 workers	defects found

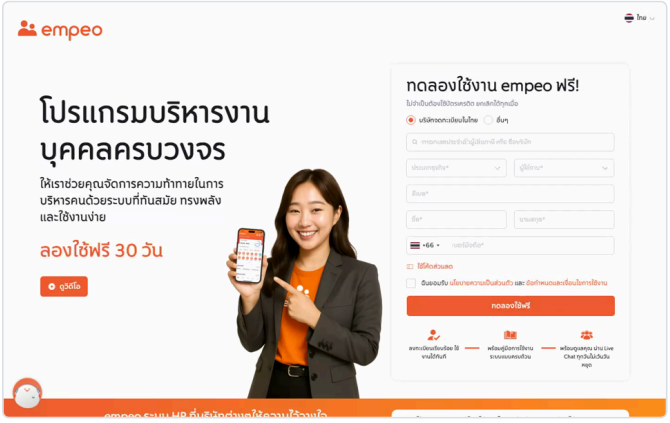
1. Submission package

Item	Location
Source repository	https://github.com/stormnoopper/gofive-empeo-registration-automation
Demo video	01_Demo/REG-E2E-01-happy-path.mp4 · storyboard in section 2
Test cases (Excel)	02_Testcase/TEST_CASES.xlsx · full list in section 3
Automation code	03_Automate_Code/ · full source in Appendix A
CI workflow	.github/workflows/registration-tests.yml

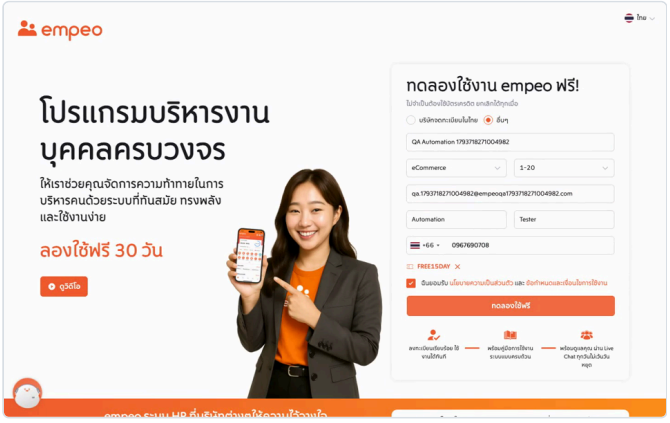
Access: the repository is private. Send me your GitHub username and I'll add you as a collaborator. Everything needed to review the work is also included in this document.

2. Demo – automated happy path

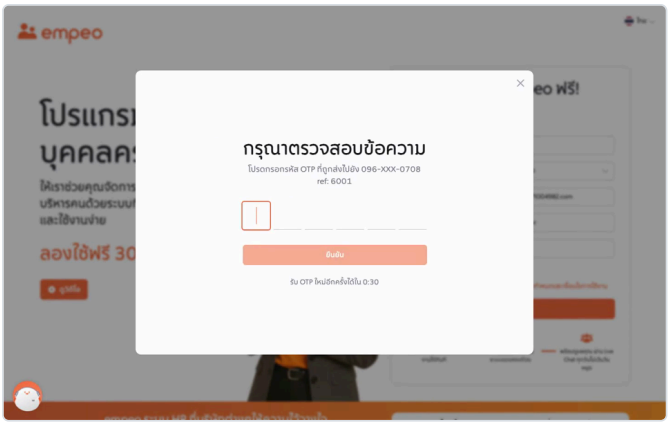
Automated run of **REG-E2E-01** (21 s, recorded by Playwright). Full video: 01_Demo/REG-E2E-01-happy-path.mp4. To record it again: `npm run test:demo`.



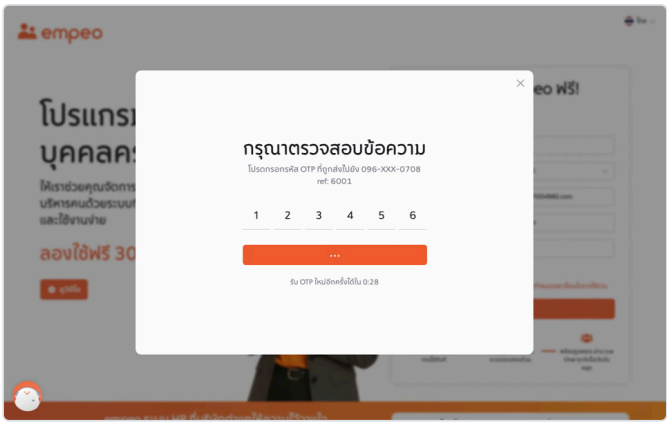
1. Registration page opened



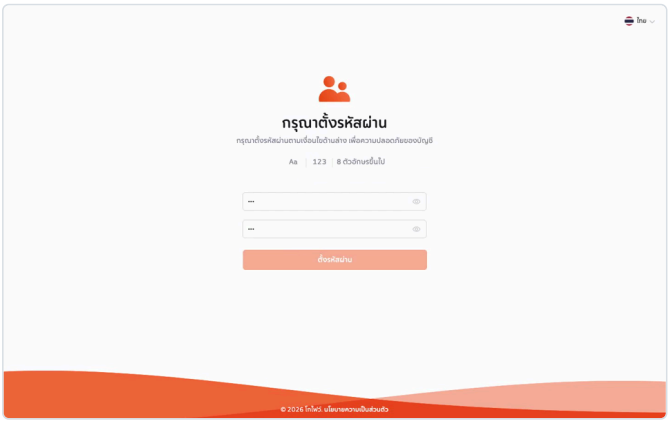
2. Form filled, promo FREE15DAY applied, consent accepted



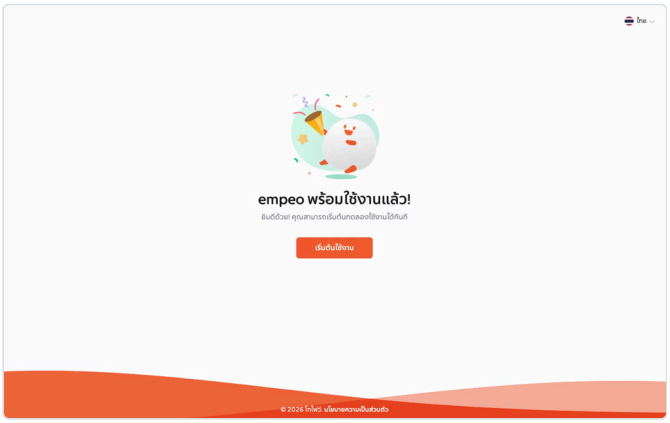
3. OTP dialog: masked phone 096-XXX-0708, ref number



4. OTP 123456 entered, verifying



5. Create password: rules Aa / 123 / 8+ gate the button



6. Success: 'empeo พร้อมใช้งานแล้ว!'

3. Test cases

Area	Tests	Scenarios
Required fields	5	Empty submit (Thai / Others), consent missing, errors clear after correction
Company type	2	Thai ↔ Others toggle, tax-ID search and selection
Email	8	Malformed ×4, valid, already registered, domain already registered, public provider
Mobile number	6	Too short, too long, wrong prefix, letters, valid with and without leading 0
Length limits	5	Email 150, first / last name 255, phone 15, company 255
Promo code	9	Valid, case-insensitive, unknown ×2, special characters, remove, max length 50, reused (stub), isValid=false (stub)
OTP	6	Dialog content, incomplete, non-numeric, wrong OTP, resend, expired (stub)
End-to-end	1	Register → OTP → password rules → success page

The full case list with steps and expected results is in `02_Testcase/TEST_CASES.xlsx`.

Test case list

TC ID	Scenario / Data	Steps	Expected result	Mode	Status
Required fields					
REG-VAL-01 Negative · High	Empty submit (Thai company) shows every required-field error -	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Keep 'บริษัทจดทะเบียนในไทย' selected 3. Click 'ทดลองใช้ฟรี'	Errors shown: กรุณากรอกเลขประจำตัวผู้เสียภาษีหรือชื่อบริษัท / กรุณากรอกประเภทธุรกิจ / กรุณากรอกผู้ใช้งาน / กรุณากรอกอีเมล / กรุณากรอกชื่อ / กรุณากรอกนามสกุล / กรุณากรอกเบอร์โทรศัพท์มือถือ / กรุณารับทราบนโยบายความเป็นส่วนตัว และข้อกำหนดและเงื่อนไขการใช้งาน. Stays on page.	Real	Pass

REG-VAL-02 Negative · High	Empty submit (Others) asks for company name -	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Select 'อื่นๆ' 3. Click 'ทดลองใช้ฟรี'	'กรุณากรอกข้อมูลที่จำเป็นทั้งหมด' and field errors are shown	Real	**Pass**
REG-VAL-03 Negative · High	All fields valid except consent blocks submission Valid registrant: company 'QA Automation <id>', business type eCommerce, users 1-20, email qa.<id>@empeoqa<id>.com, name Automation Tester, phone 0967690708	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Fill all fields with valid data 3. Leave consent unchecked 4. Click 'ทดลองใช้ฟรี'	No registration request is sent; user stays on the registration page	Real (submit guarded)	**Pass** Non-validation API calls are aborted by the test as a safety guard
REG-VAL-04 Positive · Medium	Required errors clear once valid data is entered Valid registrant: company 'QA Automation <id>', business type eCommerce, users 1-20, email qa.<id>@empeoqa<id>.com, name Automation Tester, phone 0967690708	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ทดลองใช้ฟรี' (empty) 3. Fill all fields with valid data	No validation error remains visible	Real	**Pass**
REG-VAL-05 Negative · Medium	Consent error is shown when only consent is missing Valid registrant: company 'QA Automation <id>', business type eCommerce, users 1-20, email qa.<id>@empeoqa<id>.com, name Automation Tester, phone 0967690708	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Fill all fields with valid data 3. Leave consent unchecked 4. Click 'ทดลองใช้ฟรี'	'กรุณารับทราบนโยบายความเป็นส่วนตัว และข้อกำหนดและเงื่อนไขการใช้งาน' is displayed	Real (submit guarded)	**Fail** Actual: nothing happens, no message. Defect D-01. Automated as test.fail
Company type					
REG-CMP-01 Positive · Medium	Switching company type toggles tax search and company name inputs -	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Verify tax search input is shown 3. Select 'อื่นๆ' 4. Select 'บริษัทจดทะเบียนในไทย'	Thai = tax ID/company search input; Others = company name input	Real	**Pass**
REG-CMP-02 Positive · Medium	Tax search by company name returns DBD record and fills tax ID Keyword: โกไฟว์	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Type 'โกไฟว์' in tax search 3. Select 'บริษัท โกไฟว์ จำกัด'	Tax ID field shows 0745561010054	Real	**Pass**
Email					
REG-VAL-10.1 Negative · High	Rejects malformed email (no at-sign) Email: 'abc'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out of the field	'กรุณาใส่อีเมลที่ถูกต้อง' is displayed	Real	**Pass**
REG-VAL-10.2 Negative · High	Rejects malformed email (no top-level domain) Email: 'a@b'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out of the field	'กรุณาใส่อีเมลที่ถูกต้อง' is displayed	Real	**Pass**
REG-VAL-10.3 Negative · High	Rejects malformed email (double at-sign) Email: 'user@@qa<id>.com'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out of the field	'กรุณาใส่อีเมลที่ถูกต้อง' is displayed	Real	**Pass**
REG-VAL-10.4 Negative · High	Rejects malformed email (leading space) Email: ' spaced@qa<id>.com'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out of the field	'กรุณาใส่อีเมลที่ถูกต้อง' is displayed	Real	**Pass**
REG-VAL-11 Positive · High	Accepts a well-formed, unused email Email: qa.<id>@empeoqa<id>.com	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out	No invalid / duplicate error	Real	**Pass**
REG-VAL-12 Negative · High	Rejects an email that already has an account Email: admin@gofive.co.th	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out	'อีเมลนี้มีบัญชีใช้งานแล้ว ลองล็อกอินหรือทำเรื่องลืมรหัสผ่าน' is displayed	Real	**Pass**

REG-VAL-13 Negative · Medium	Rejects a new email on a company domain that is already registered Email: qa.<id>@mailinator.com	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out	'โดเมนนี้มีบัญชีใช้งานแล้ว' is displayed	Real	Pass
REG-VAL-14 Positive · Medium	Accepts a new email on a public mail provider domain Email: qa.<id>@gmail.com	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter email 3. Move focus out	No error (public domains are allowed)	Real	Pass
Mobile number					
REG-VAL-20.1 Negative · High	Rejects invalid mobile number (9 digits) Phone: 096769070	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	'รูปแบบเบอร์โทรศัพท์ไม่ถูกต้อง' is displayed	Real	Pass
REG-VAL-20.2 Negative · High	Rejects invalid mobile number (11 digits) Phone: 09676907081	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	'รูปแบบเบอร์โทรศัพท์ไม่ถูกต้อง' is displayed	Real	Pass
REG-VAL-20.3 Negative · High	Rejects invalid mobile number (invalid prefix) Phone: 1234567890	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	'รูปแบบเบอร์โทรศัพท์ไม่ถูกต้อง' is displayed	Real	Pass
REG-VAL-20.4 Negative · High	Rejects invalid mobile number (letters) Phone: abcdef	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	'รูปแบบเบอร์โทรศัพท์ไม่ถูกต้อง' is displayed	Real	Pass
REG-VAL-21.1 Positive · High	Accepts valid Thai mobile number (with leading 0) Phone: 0967690708	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	No phone format error	Real	Pass
REG-VAL-21.2 Positive · High	Accepts valid Thai mobile number (without leading 0) Phone: 967690708	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter mobile number 3. Move focus out	No phone format error	Real	Pass
Field length limits					
REG-VAL-30.1 Edge · Low	Email is capped at 150 characters 300 characters 'a'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter 300 characters into Email	Field value is truncated to 150 characters	Real	Pass
REG-VAL-30.2 Edge · Low	First name is capped at 255 characters 300 characters 'a'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter 300 characters into First name	Field value is truncated to 255 characters	Real	Pass
REG-VAL-30.3 Edge · Low	Last name is capped at 255 characters 300 characters 'a'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter 300 characters into Last name	Field value is truncated to 255 characters	Real	Pass
REG-VAL-30.4 Edge · Low	Mobile number is capped at 15 characters 300 characters 'a'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Enter 300 characters into Mobile number	Field value is truncated to 15 characters	Real	Pass
REG-VAL-31 Edge · Low	Company name (Others) is capped at 255 characters 300 characters 'a'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Select 'อื่นๆ' 3. Enter 300 characters into company name	Value truncated to 255 characters	Real	Pass
Promo code					
REG-PRM-01 Positive · High	Valid promo code is accepted and shown as applied Promo: FREE15DAY	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter code 4. Click 'ใช้โค้ด'	API returns isValid=true; code shown as applied with remove (✕) icon	Real	Pass
REG-PRM-02 Positive · Medium	Promo code is case-insensitive Promo: free15day	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter lowercase code 4. Click 'ใช้โค้ด'	Accepted; API normalises to FREE15DAY	Real	Pass

REG-PRM-03.1 Negative · High	Unknown promo code is rejected Promo: INVALID99	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter code 4. Click 'ใช้โค้ด'	HTTP 400; 'คูปองโค้ดไม่ถูกต้อง' is displayed	Real	**Pass**
REG-PRM-03.2 Negative · Medium	Near-match promo code is rejected Promo: FREE15DAYX	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter code 4. Click 'ใช้โค้ด'	HTTP 400; 'คูปองโค้ดไม่ถูกต้อง' is displayed	Real	**Pass**
REG-PRM-04 Positive · Medium	Applied promo code can be removed Promo: FREE15DAY	1. Apply FREE15DAY 2. Click × next to the code	Code removed; promo input shown again	Real	**Pass**
REG-PRM-05 Edge · Low	Promo code input is capped at 50 characters 80 characters 'X'	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter 80 characters	Value truncated to 50 characters	Real	**Pass**
REG-PRM-06 Negative · High	Exhausted / reused promo code is rejected Promo: USEDCODE01	1. Stub coupon-validation → 400 2. Enter code 3. Click 'ใช้โค้ด'	'คูปองโค้ดไม่ถูกต้อง' is displayed; code not applied	Stub	**Pass** No exhausted code available as test data
REG-PRM-07 Negative · Medium	Coupon flagged isValid=false must not be applied Promo: USEDCODE02	1. Stub coupon-validation → 200, isValid=false 2. Enter code 3. Click 'ใช้โค้ด'	Code is not shown as applied	Stub	**Fail** Actual: code is shown as applied. Defect D-02. Automated as test.fail
REG-PRM-08 Negative · Low	Special-character promo code is rejected without applying Promo: !@#$%	1. Open https://portal.uat.gofive.co.th/Register/empeo 2. Click 'ใช้โค้ดส่วนลด' 3. Enter code 4. Click 'ใช้โค้ด'	Code not applied; promo input still shown	Real	**Pass**
End-to-end					
REG-E2E-01 Positive · Critical	Register with promo code, verify OTP and set password Valid registrant: company 'QA Automation <id>', business type eCommerce, users 1-20, email qa.<id>@empeoqa<id>.com, name Automation Tester, phone 0967690708; promo FREE15DAY; OTP 123456; password EmpeoQa2026	1. Fill all fields + apply FREE15DAY + accept consent 2. Click 'ทดลองใช้ฟรี' 3. Enter OTP 123456 → 'ยืนยัน' 4. On create-password: try 'abc' and lowercase password 5. Enter valid password + confirm → 'ตั้งรหัสผ่าน'	Register API payload matches input (phone +66967690708, couponCode FREE15DAY); OTP isValid=true; button disabled until rules Aa / 123 / 8+ are met; 'empeo พร้อมใช้งานแล้ว!' and 'เริ่มต้นใช้งาน' shown	Real	**Pass** Video: demo/REG-E2E-01-happy-path.mp4
OTP					
REG-OTP-01 Positive · Medium	Dialog shows masked phone, reference and countdown Phone: 0967690708	1. Submit a valid registration 2. Observe OTP dialog	'กรุณาตรวจสอบข้อความ', 'โปรดกรอกรหัส OTP ที่ถูกส่งไปยัง 096-XXX-0708', 'ref: <n>' and resend countdown shown; resend link hidden	Real	**Pass**
REG-OTP-02 Negative · Medium	Verify button stays disabled until all 6 digits are entered OTP: 12345 → 123456	1. Enter 5 digits 2. Enter the 6th digit	Verify disabled with 5 digits, enabled with 6	Real	**Pass**
REG-OTP-03 Negative · Medium	Non-numeric characters are not accepted OTP: abcdef	1. Type 'abcdef' into OTP inputs	OTP inputs stay empty; verify disabled	Real	**Pass**
REG-OTP-04 Negative · High	Wrong OTP is rejected and inputs are cleared OTP: 000000	1. Enter 000000 2. Click 'ยืนยัน'	'OTP ไม่ถูกต้อง กรุณาลองใหม่อีกครั้ง' shown; inputs cleared; stays on registration page	Real	**Pass**

REG-OTP-05 Positive · Medium	Resend issues a new reference and restarts the countdown -	1. Wait for countdown to finish 2. Click 'ส่งใหม่อีกครั้ง'	resend-otp API succeeds; ref number changes; countdown restarts	Real	**Pass**
REG-OTP-06 Negative · High	Expired OTP shows an error and keeps the dialog open OTP: 123456	1. Stub verify-otp → expired 2. Enter OTP → 'ยืนยัน' 3. Click 'OK' on popup	Error popup shown; after dismissal the OTP dialog is still open	Stub	**Pass** Popup text is generic 'Business error with 1 errors.' – see D-03

4. Defects found

ID	Severity	Title	Steps	Expected	Actual
D-01	Medium	No feedback when only the consent checkbox is missing REG-VAL-05	1. Fill every field with valid data 2. Leave consent unchecked 3. Click 'ทดลองใช้ฟรี'	Consent error message is displayed	Nothing happens: no message, no request
D-02	Medium	Coupon applied even when API returns isValid=false REG-PRM-07	1. coupon-validation returns 200 with isValid=false 2. Apply the code	Code is rejected / not applied	Code is shown as applied (verified with stubbed response)
D-03	Low	Raw backend error shown for OTP business errors REG-OTP-06	1. verify-otp returns business error (e.g. E4002 expired)	User-friendly message such as 'OTP หมดอายุ กรุณาขอรหัสใหม่'	Popup 'Error! Business error with 1 errors.'
D-04	Low	Submit enabled when password and confirmation differ REG-E2E-01	1. Reach create-password page 2. Password 'Automation123', confirm 'Automation124'	Button disabled or mismatch error shown	Button 'ตั้งรหัสผ่าน' is enabled (found in exploration, not submitted)

5. Prerequisites

Item	Requirement
Node.js	v20 or newer (tested on v22). Check with `node -v`
npm	Comes with Node.js
OS	macOS, Windows or Linux
Network	Access to `portal.uat.gofive.co.th` and `portal-uat-api.gofive.co.th`

6. Installation

1. Clone the repository and open the code folder.

```
git clone https://github.com/stormnoopper/gofive-empeo-registration-automation.git
cd gofive-empeo-registration-automation/03_Automate_Code
```

2. Install the dependencies. Versions are pinned in `package-lock.json`.

```
npm ci
```

3. Download the Chromium browser used by Playwright.

```
npx playwright install chromium
```

4. Optional: confirm the project compiles.

```
npm run typecheck
```

7. Running the tests

Command	What it does
`npm test`	Full suite: form validation, promo code, OTP and end-to-end registration
`npm run test:ui-only`	Everything except live submissions (`@e2e` excluded). Creates **no** data on UAT
`npm run test:e2e`	Only the live registration, OTP and create-password tests
`npm run test:smoke`	Critical path only (`@smoke`): empty submit, valid promo, full happy path
`npm run test:headed`	Full suite in a visible browser, one test at a time
`npm run test:demo`	Smoke tests in a visible browser, slowed down, with video recorded

Run a single test or group

```
npx playwright test -g "REG-E2E-01"          # by test case ID
npx playwright test tests/promo-code.spec.ts # by file
npx playwright test --ui                     # interactive UI mode
```

Note: each full run (`npm test`) creates 7 new trial registrations on UAT. Company name and email domain are unique per run, so the suite can be re-run any number of times without data clean-up. Use `npm run test:ui-only` when you do not want to create any UAT data.

8. Viewing results

HTML report

```
npm run report
```

This opens `playwright-report/index.html`. Every test case ID is shown with its status, duration and steps. A failing test also includes its screenshot, video and trace.

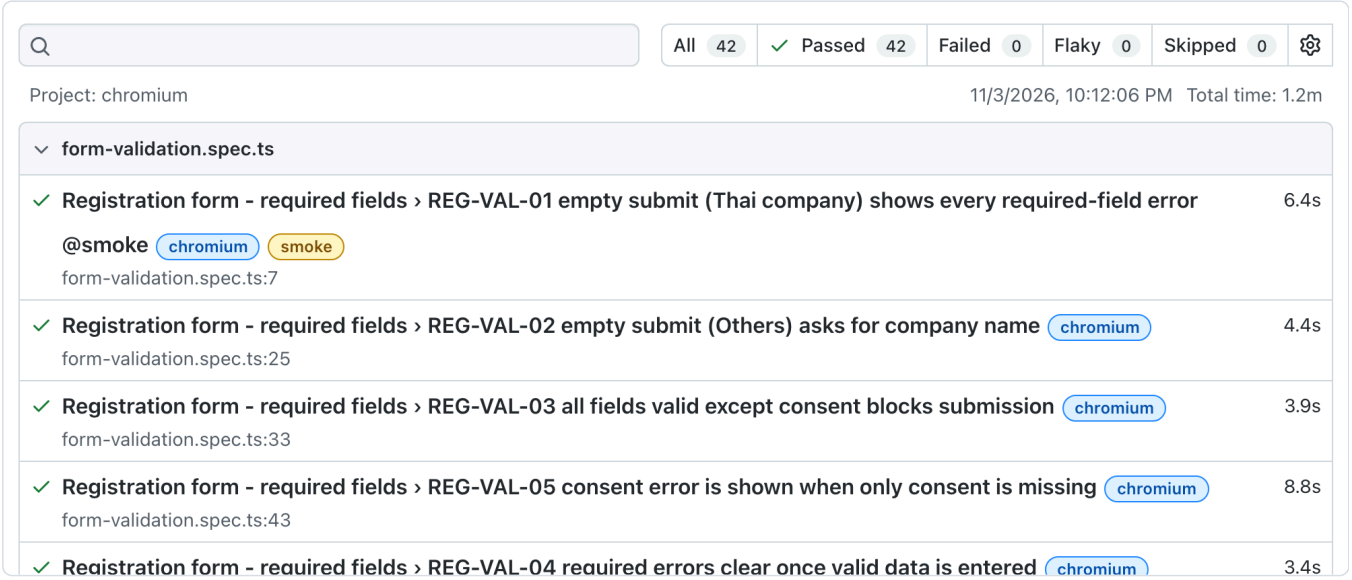


Figure 1 – Playwright HTML report: 42 passed, 0 failed, 0 flaky.

Debugging a failure

```
npx playwright show-trace test-results/<test-folder>/trace.zip
```

The trace viewer replays every action with DOM snapshots, network calls and console logs. A JUnit file for CI tools is written to `test-results/junit.xml`.

9. Configuration

Every value has a default, so nothing needs to be configured to run. To override one, set an environment variable, for example `TEST_PROMO=OTHERCODE npm test`.

Variable	Default	Purpose
`BASE_URL`	https://portal.uat.gofive.co.th	Web app under test
`API_BASE_URL`	https://portal-uat-api.gofive.co.th	Backend host used for request guards
`TEST_PHONE`	0967690708	Fixed mobile number
`TEST_OTP`	123456	Fixed OTP
`TEST_PROMO`	FREE15DAY	Valid promo code
`TEST_PASSWORD`	EmpeoQa2026	Password set on the create-password page
`REGISTERED_EMAIL`	admin@gofive.co.th	Existing account for the duplicate-email test
`REGISTERED_DOMAIN`	mailinator.com	Existing domain for the duplicate-domain test
`RECORD_VIDEO`	unset	`1` = record video for every test

10. Project structure

```
03_Automate_Code/
├── src/
│   ├── config/env.ts              environment and fixed test data
│   ├── data/messages.ts           expected UI messages (text as constants)
│   ├── data/registrant.ts         unique registrant factory, phone helpers
│   ├── fixtures/test.ts           custom fixtures, third-party traffic blocking
│   └── pages/
│       ├── RegisterPage.ts        registration form
│       ├── OtpDialog.ts           OTP dialog
│       └── CreatePasswordPage.ts  create password and success page
├── tests/
│   ├── form-validation.spec.ts    required fields, email, phone, limits, company type
│   ├── promo-code.spec.ts         valid / invalid / case / remove / reused
│   └── registration-e2e.spec.ts   happy path and OTP scenarios (@e2e)
├── .github/workflows/registration-tests.yml
└── playwright.config.ts
```

11. Reliability design

- **Unique data per run:** the backend rejects reused emails and reused company email domains, so every registrant gets a fresh company name and domain.
- **Stable locators:** the app's `data-testid` attributes are used everywhere. No index or CSS-path locators.
- **No fixed waits:** the suite waits on real signals: API responses, the form leaving `ng-pending`, URL changes, and web-first assertions.

- **Guarded negative submits:** tests that must not register abort every non-validation backend call, so a client-side bug cannot silently create data.
- **Third-party traffic blocked:** analytics, ads and heatmap scripts are aborted, which makes runs faster and steadier.
- **Stubs only where needed:** expired OTP and exhausted promo code cannot be reproduced on demand. All other tests use the live UAT API.
- **Known defects tracked:** confirmed defects are marked `test.fail`, so CI stays green. When a fix ships, the test turns red to signal that the marker should be removed.
- **CI-ready:** on push and PR, GitHub Actions runs the `ui-only` suite, which creates no UAT data. The full suite runs via *Run workflow → full*. Both use 2 retries and publish HTML and JUnit reports as artifacts. Add a `schedule` trigger for a nightly run.

12. Troubleshooting

Symptom	Fix
`Executable doesn't exist`	Run `npx playwright install chromium`
Timeouts on OTP verification	UAT verification can take up to ~40 s. The suite allows 90 s. Re-run, or check UAT availability.
Duplicate email / domain errors	Make sure `REGISTERED_EMAIL` / `REGISTERED_DOMAIN` still exist on UAT, or override them.
Tests fail behind a proxy or VPN	Allow both UAT hosts listed in section 1.

13. Conclusion

42	**40**	**2**	**0**
test cases automated	pass as expected	fail = confirmed defects	flaky across repeated runs

Result

The empeo registration flow works end to end on UAT. A new company can register with the promo code `FREE15DAY`, verify OTP `123456`, set a password and reach "empeo พร้อมใช้งานแล้ว!". Field validation covers required fields, email format, duplicate email/domain, Thai mobile format and field length limits, and it behaves correctly.

Coverage against the exam scope

Requested scope	Covered	Test cases
Successful registration with valid data	Yes	REG-E2E-01
OTP verification flow	Yes	REG-OTP-01 to 05, REG-E2E-01
Promo code validation	Yes	REG-PRM-01 to 08
Required fields and error handling	Yes	REG-VAL-01 to 05, REG-CMP-01/02
Invalid phone numbers	Yes	REG-VAL-20/21
Expired OTP	Yes (stub)	REG-OTP-06
Reused promo code	Yes (stub)	REG-PRM-06/07

Key findings

- Four defects found. Two are Medium: the consent checkbox gives no feedback when it is the only thing missing, and the coupon UI trusts the HTTP status over `isValid`. Two are Low: a raw backend error appears on OTP failure, and password/confirmation mismatch is not blocked.
- The backend blocks duplicate company **email domains**, not only duplicate emails. Test data must therefore use a fresh domain on every run.

- Correct-OTP verification on UAT sometimes takes ~40 s. This may be worth monitoring as a performance item.

Limitations and next steps

- The Thai-company end-to-end submit is not automated, because each real tax ID can be registered only once. Recommendation: provide a resettable test tax ID or a data clean-up API on UAT.
- Expired OTP and exhausted promo code are covered by API stubs. Recommendation: provide a short-TTL OTP setting or a dedicated exhausted coupon on UAT, so these can run against the real backend.
- Possible extensions: run cross-browser (Firefox / WebKit / mobile viewport), cover the English locale, and add API-level contract tests for `/api/v2/register`, `verify-otp` and `coupon-validation`.

Appendix A – Source code

Complete source of the automation project, as committed in the repository.

.github/workflows/registration-tests.yml 49 lines

```
name: empeo registration tests

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
    inputs:
      suite:
        description: 'Suite to run'
        type: choice
        options: [ui-only, full]
        default: ui-only

concurrency:
  group: empeo-registration-${{ github.ref }}
  cancel-in-progress: true

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    defaults:
      run:
        working-directory: 03_Automate_Code
    env:
      CI: 'true'
      TZ: Asia/Bangkok
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: 03_Automate_Code/package-lock.json
      - run: npm ci
      - run: npx playwright install --with-deps chromium
      - run: npm run typecheck
      - name: Run tests
        run: |
          if [ "${{ inputs.suite }}" = "full" ]; then npm test; else npm run test:ui-only; fi
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}
        with:
          name: playwright-report
          path: |
            03_Automate_Code/playwright-report/
            03_Automate_Code/test-results/
          retention-days: 14
```

03_Automate_Code/package.json 21 lines

```
{
  "name": "empeo-registration-automation",
  "version": "1.0.0",
  "private": true,
  "description": "Playwright automation suite for the empeo registration flow (Gofive Automation Tester
Exam 1.0)",
  "scripts": {
    "test": "playwright test",
    "test:ui-only": "playwright test --grep-invert @e2e",
    "test:e2e": "playwright test --grep @e2e",
    "test:smoke": "playwright test --grep @smoke",
    "test:headed": "playwright test --headed --workers=1",
```

```
    "test:demo": "RECORD_VIDEO=1 playwright test --headed --workers=1 --grep @smoke",
    "report": "playwright show-report",
    "typecheck": "tsc --noEmit"
  },
  "devDependencies": {
    "@playwright/test": "1.56.1",
    "@types/node": "22.10.2",
    "typescript": "5.6.3"
  }
}
```

03_Automate_Code/playwright.config.ts 36 lines

```
import { defineConfig, devices } from '@playwright/test';
import { env } from './src/config/env';

const isCI = !!process.env.CI;

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,
  forbidOnly: isCI,
  retries: isCI ? 2 : 1,
  workers: isCI ? 2 : 4,
  timeout: 60_000,
  expect: { timeout: 10_000 },
  reporter: [
    ['list'],
    ['html', { open: 'never' }],
    ['junit', { outputFile: 'test-results/junit.xml' }],
  ],
  use: {
    baseURL: env.baseUrl,
    locale: 'th-TH',
    timezoneId: 'Asia/Bangkok',
    actionTimeout: 15_000,
    navigationTimeout: 45_000,
    trace: 'retain-on-failure',
    screenshot: 'only-on-failure',
    video: { mode: env.recordVideo ? 'on' : 'retain-on-failure', size: { width: 1440, height: 900 } },
    launchOptions: { slowMo: env.recordVideo ? 250 : 0 },
  },
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'], viewport: { width: 1440, height: 900 } },
    },
  ],
});
```

03_Automate_Code/tsconfig.json 14 lines

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "commonjs",
    "moduleResolution": "node",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "noEmit": true,
    "baseUrl": ".",
    "paths": { "@src/*": ["src/*"] }
  },
  "include": ["src", "tests", "playwright.config.ts"]
}
```

03_Automate_Code/src/config/env.ts 11 lines

```
export const env = {
  baseUrl: process.env.BASE_URL ?? 'https://portal.uat.gofive.co.th',
```

```
  apiBaseUrl: process.env.API_BASE_URL ?? 'https://portal-uat-api.gofive.co.th',
  phone: process.env.TEST_PHONE ?? '0967690708',
  otp: process.env.TEST_OTP ?? '123456',
  password: process.env.TEST_PASSWORD ?? 'EmpeoQa2026',
  promoCode: process.env.TEST_PROMO ?? 'FREE15DAY',
  registeredDomain: process.env.REGISTERED_DOMAIN ?? 'mailinator.com',
  registeredEmail: process.env.REGISTERED_EMAIL ?? 'admin@gofive.co.th',
  recordVideo: process.env.RECORD_VIDEO === '1',
} as const;
```

03_Automate_Code/src/data/messages.ts 27 lines

```
export const Messages = {
  required: {
    taxIdOrCompany: 'กรุณากรอกเลขประจำตัวผู้เสียภาษีหรือชื่อบริษัท',
    allRequired: 'กรุณากรอกข้อมูลที่จำเป็นทั้งหมด',
    businessType: 'กรุณากรอกประเภทธุรกิจ',
    userAmount: 'กรุณากรอกผู้ใช้งาน',
    email: 'กรุณากรอกอีเมล',
    firstName: 'กรุณากรอกชื่อ',
    lastName: 'กรุณากรอกนามสกุล',
    phone: 'กรุณากรอกเบอร์โทรศัพท์มือถือ',
    consent: 'กรุณารับทราบนโยบายความเป็นส่วนตัว และข้อกำหนดและเงื่อนไขการใช้งาน',
  },
  invalid: {
    email: 'กรุณาใส่อีเมลที่ถูกต้อง',
    phone: 'รูปแบบเบอร์โทรศัพท์ไม่ถูกต้อง',
    coupon: 'คูปองโค้ดไม่ถูกต้อง',
  },
  otp: {
    title: 'กรุณาตรวจสอบข้อความ',
    invalid: 'OTP ไม่ถูกต้อง กรุณาลองใหม่อีกครั้ง',
    sentTo: (maskedPhone: string) => `โปรดกรอกรหัส OTP ที่ถูกส่งไปยัง ${maskedPhone}`,
  },
  duplicate: {
    email: 'อีเมลนี้มีบัญชีใช้งานแล้ว ลองล็อกอินหรือทำเรื่องลืมรหัสผ่าน',
    domain: 'โดเมนนี้มีบัญชีใช้งานแล้ว',
  },
} as const;
```

03_Automate_Code/src/data/registrant.ts 39 lines

```
import { env } from '../config/env';

export type CompanyType = 'thai' | 'others';

export interface Registrant {
  companyType: CompanyType;
  company: string;
  businessType: string;
  userAmount: string;
  email: string;
  firstName: string;
  lastName: string;
  phone: string;
  promoCode?: string;
}


export const uniqueId = () => `${Date.now()}${Math.floor(Math.random() * 1_000).toString().padStart(3,
'0')}`;

export const uniqueEmail = (id = uniqueId()) => `qa.${id}@empeoqa${id}.com`;

export const toE164 = (phone: string) => `+66${phone.replace(/^0/, '')}`;

export const maskPhone = (phone: string) => `${phone.slice(0, 3)}-XXX-${phone.slice(-4)}`;

export const buildRegistrant = (overrides: Partial<Registrant> = {}): Registrant => {
  const id = uniqueId();
  return {
```

```
    companyType: 'others',
    company: `QA Automation ${id}`,
    businessType: 'eCommerce',
    userAmount: '1-20',
    email: uniqueEmail(id),
    firstName: 'Automation',
    lastName: 'Tester',
    phone: env.phone,
    ...overrides,
  };
};
```

03_Automate_Code/src/fixtures/test.ts 38 lines

```
import { test as base, expect } from '@playwright/test';
import { RegisterPage } from '../pages/RegisterPage';
import { OtpDialog } from '../pages/OtpDialog';
import { CreatePasswordPage } from '../pages/CreatePasswordPage';
import { env } from '../config/env';

const THIRD_PARTY =
  /google-
analytics\.com|googletagmanager\.com|googlesyndication\.com|doubleclick\.net|googleadservices\.com|facebook\.
(com|net)|heapanalytics\.com|hotjar|clarity\.ms|tiktok|zdassets|zopim|youtube\.com/i;

type Fixtures = {
  registerPage: RegisterPage;
  otpDialog: OtpDialog;
  createPasswordPage: CreatePasswordPage;
};

export const test = base.extend<Fixtures>({
  context: async ({ context }, use) => {
    await context.route(THIRD_PARTY, (route) => route.abort());
    await use(context);
  },
  page: async ({ page }, use) => {
    await use(page);
    if (env.recordVideo) await page.waitForTimeout(2_500);
  },
  registerPage: async ({ page }, use) => {
    const registerPage = new RegisterPage(page);
    await registerPage.open();
    await use(registerPage);
  },
  otpDialog: async ({ page }, use) => {
    await use(new OtpDialog(page));
  },
  createPasswordPage: async ({ page }, use) => {
    await use(new CreatePasswordPage(page));
  },
});

export { expect };
```

03_Automate_Code/src/pages/RegisterPage.ts 186 lines

```
import { expect, type Locator, type Page, type Response } from '@playwright/test';
import type { CompanyType, Registrant } from '../data/registrant';
import { env } from '../config/env';

const Api = {
  masterData: /\/api\/register\/additional-information/i,
  register: /\/api\/v2\/register$/i,
  duplicateEmail: /\/api\/v2\/register\/DuplicateEmail/i,
  couponValidation: /\/api\/Register\/coupon-validation/i,
  taxSearch: /\/api\/register\/tax-search/i,
  backend: `${env.apiBaseUrl}/api/**`,
  validators: /coupon-validation|DuplicateEmail|duplicateInfo/i,
} as const;
```

```
export class RegisterPage {
  static readonly path = '/Register/empeo';
  static readonly api = Api;

  readonly form: Locator;
  readonly thaiCompanyRadio: Locator;
  readonly otherCompanyRadio: Locator;
  readonly taxSearchInput: Locator;
  readonly companyNameInput: Locator;
  readonly businessTypeDropdown: Locator;
  readonly userAmountDropdown: Locator;
  readonly emailInput: Locator;
  readonly firstNameInput: Locator;
  readonly lastNameInput: Locator;
  readonly phoneInput: Locator;
  readonly couponToggle: Locator;
  readonly couponInput: Locator;
  readonly couponApplyButton: Locator;
  readonly couponBox: Locator;
  readonly couponRemoveButton: Locator;
  readonly consentCheckbox: Locator;
  readonly submitButton: Locator;

  constructor(readonly page: Page) {
    this.form = page.locator('form');
    this.thaiCompanyRadio = page.getByTestId('input_radio_registration_company_thai');
    this.otherCompanyRadio = page.getByTestId('input_radio_registration_company_others');
    this.taxSearchInput = page.getByTestId('input_textfield_input_registration_tax_id');
    this.companyNameInput = page.getByTestId('input_textfield_input_register_company_name');
    this.businessTypeDropdown = page.getByTestId('dropdown_selection_registration_company_type');
    this.userAmountDropdown = page.getByTestId('dropdown_selection_registration_user_amount');
    this.emailInput = page.getByTestId('input_textfield_input_registration_email');
    this.firstNameInput = page.getByTestId('input_textfield_input_register_first_name');
    this.lastNameInput = page.getByTestId('input_textfield_input_register_last_name');
    this.phoneInput = page.locator('input#phone');
    this.couponToggle = page.getByTestId('input_button_registration_register_ticket_got_coupon');
    this.couponInput = page.getByTestId('input_text_registration_coupon_code');
    this.couponApplyButton = page.getByTestId('input_button_registration_btn_apply');
    this.couponBox = page.locator('.form-coupon-code');
    this.couponRemoveButton = this.couponBox.locator('.gf-icon-close');
    this.consentCheckbox = page.getByTestId('input_checkbox_registration_checkbox');
    this.submitButton = page.getByTestId('button_submit_registration_try_for_free');
  }

  async open() {
    const masterData = this.page.waitForResponse((r) => Api.masterData.test(r.url()) && r.ok());
    await this.page.goto(RegisterPage.path);
    await masterData;
    await expect(this.emailInput).toBeEditable();
  }

  error(message: string): Locator {
    return this.form.getByText(message, { exact: true }).filter({ visible: true });
  }

  visibleErrors(): Locator {
    return this.form.locator('.e-error, [data-testid="error_message_text"]').filter({ visible: true });
  }

  dropdownOption(text: string): Locator {
    return this.page.locator('go5-dropdown-item').filter({ visible: true }).getByText(text, { exact: true
});
  }

  appliedCoupon(code: string): Locator {
    return this.couponBox.getByText(code, { exact: true });
  }

  async chooseCompanyType(type: CompanyType) {
    if (type === 'others') {
      await this.otherCompanyRadio.click();
      await expect(this.companyNameInput).toBeVisible();
```

```
    } else {
      await this.thaiCompanyRadio.click();
      await expect(this.taxSearchInput).toBeVisible();
    }
  }

  async searchTaxId(keyword: string) {
    const search = this.page.waitForResponse((r) => Api.taxSearch.test(r.url()));
    await this.taxSearchInput.fill(keyword);
    await search;
  }

  async selectFirstTaxResult(companyName: string) {
    await this.page.locator('go5-dropdown-item').filter({ hasText: companyName }).first().click();
  }

  async selectBusinessType(text: string) {
    await this.businessTypeDropdown.click();
    await this.dropdownOption(text).click();
    await expect(this.businessTypeDropdown).toContainText(text);
  }

  async selectUserAmount(text: string) {
    await this.userAmountDropdown.click();
    await this.dropdownOption(text).click();
    await expect(this.userAmountDropdown).toContainText(text);
  }

  async fillEmail(email: string) {
    const check = this.page.waitForResponse((r) => Api.duplicateEmail.test(r.url()));
    await this.emailInput.fill(email);
    await this.emailInput.blur();
    await check;
  }

  async fillPhone(phone: string) {
    await this.phoneInput.fill(phone);
    await this.phoneInput.blur();
  }

  async openCouponInput() {
    if (await this.couponToggle.isVisible()) await this.couponToggle.click();
    await expect(this.couponInput).toBeVisible();
  }

  async applyCoupon(code: string) {
    await this.openCouponInput();
    await this.couponInput.fill(code);
    const validation = this.page.waitForResponse((r) => Api.couponValidation.test(r.url()));
    await this.couponApplyButton.click();
    return validation;
  }

  async removeCoupon() {
    await this.couponRemoveButton.click();
  }

  async acceptConsent() {
    await this.consentCheckbox.check();
  }

  async fill(registrant: Registrant, { acceptConsent = true } = {}) {
    await this.chooseCompanyType(registrant.companyType);
    if (registrant.companyType === 'others') {
      await this.companyNameInput.fill(registrant.company);
    } else {
      await this.searchTaxId(registrant.company);
      await this.selectFirstTaxResult(registrant.company);
    }
    await this.selectBusinessType(registrant.businessType);
    await this.selectUserAmount(registrant.userAmount);
    await this.fillEmail(registrant.email);
```

```
    await this.firstNameInput.fill(registrant.firstName);
    await this.lastNameInput.fill(registrant.lastName);
    await this.fillPhone(registrant.phone);
    if (registrant.promoCode) await this.applyCoupon(registrant.promoCode);
    if (acceptConsent) await this.acceptConsent();
  }

  blockSubmissions(): string[] {
    const blocked: string[] = [];
    void this.page.route(Api.backend, (route) => {
      const request = route.request();
      if (request.method() === 'GET' || Api.validators.test(request.url())) return route.continue();
      blocked.push(`${request.method()} ${request.url()}`);
      return route.abort();
    });
    return blocked;
  }

  async submit() {
    await expect(this.form).not.toHaveClass(/ng-pending/);
    await this.submitButton.click();
  }

  async submitAndWaitForRegistration(): Promise<Response> {
    const registration = this.page.waitForResponse((r) => Api.register.test(r.url()) &&
r.request().method() === 'POST');
    await this.submit();
    return registration;
  }
}
```

03_Automate_Code/src/pages/OtpDialog.ts 67 lines

```
import { expect, type Locator, type Page, type Response } from '@playwright/test';

const Api = {
  verifyOtp: /\/api\/v2\/register\/verify-otp/i,
  resendOtp: /\/api\/register\/empeo\/resend-otp/i,
} as const;

export class OtpDialog {
  static readonly api = Api;

  readonly root: Locator;
  readonly digits: Locator;
  readonly verifyButton: Locator;
  readonly resendButton: Locator;
  readonly countdown: Locator;
  readonly reference: Locator;
  readonly errorPopup: Locator;
  readonly errorPopupConfirm: Locator;

  constructor(readonly page: Page) {
    this.root = page.locator('go5-dialog').filter({ has:
page.getByTestId('input_text_registration_otp_Config') });
    this.digits = page.getByTestId('input_text_registration_otp_Config').locator('input');
    this.verifyButton = page.getByTestId('button_button_registration_verify');
    this.resendButton = page.getByTestId('button_button_registration_otp_resend');
    this.countdown = this.root.getByText(/รับ OTP ใหม่อีกครั้งได้ใน \d+:\d{2}/);
    this.reference = this.root.getByText(/ref:\s*\d+/);
    this.errorPopup = page.locator('.swal2-popup');
    this.errorPopupConfirm = page.locator('.swal2-confirm');
  }

  async expectOpen() {
    await expect(this.digits).toHaveCount(6);
    await expect(this.digits.first()).toBeVisible();
  }

  async referenceId(): Promise<string> {
    return (await this.reference.innerText()).match(/ref:\s*(\d+)/)?.[1] ?? '';
  }
```

```
  async enter(code: string) {
    await this.digits.first().click();
    await this.page.keyboard.type(code);
  }

  async value(): Promise<string> {
    return (await this.digits.evaluateAll((inputs) => inputs.map((input) => (input as
HTMLInputElement).value))).join('');
  }

  async verify(code: string): Promise<Response> {
    await this.enter(code);
    const response = this.page.waitForResponse((r) => Api.verifyOtp.test(r.url()), { timeout: 90_000 });
    await this.verifyButton.click();
    return response;
  }

  async resend(): Promise<Response> {
    await expect(this.resendButton).toBeVisible({ timeout: 45_000 });
    const response = this.page.waitForResponse((r) => Api.resendOtp.test(r.url()));
    await this.resendButton.click();
    return response;
  }

  async dismissErrorPopup() {
    await this.errorPopupConfirm.click();
    await expect(this.errorPopup).toBeHidden();
  }
}
```

03_Automate_Code/src/pages/CreatePasswordPage.ts 45 lines

```
import { expect, type Locator, type Page } from '@playwright/test';

export type PasswordRule = 'Aa' | '123' | '8 ตัวอักษรขึ้นไป';

export class CreatePasswordPage {
  static readonly url = /\/Register\/empeo\/create-password\?token=/;

  readonly passwordInput: Locator;
  readonly confirmPasswordInput: Locator;
  readonly submitButton: Locator;
  readonly successHeading: Locator;
  readonly startButton: Locator;

  constructor(readonly page: Page) {
    this.passwordInput = page.getByTestId('input_textfield_input_input_password_crate_password_password');
    this.confirmPasswordInput =
page.getByTestId('input_textfield_input_input_password_crate_password_confirm_password');
    this.submitButton = page.getByTestId('button_button_create_password_btn_next');
    this.successHeading = page.getByText('empeo พร้อมใช้งานแล้ว!', { exact: true });
    this.startButton = page.getByText('เริ่มต้นใช้งาน', { exact: true });
  }

  rule(name: PasswordRule): Locator {
    return this.page.locator('.condition-item').filter({ hasText: name });
  }

  ruleSatisfied(name: PasswordRule): Locator {
    return this.rule(name).locator('.gf-icon-checkbox');
  }

  async waitForPage() {
    await expect(this.page).toHaveURL(CreatePasswordPage.url, { timeout: 90_000 });
    await expect(this.passwordInput).toBeEditable();
  }

  async fill(password: string, confirm = password) {
    await this.passwordInput.fill(password);
    await this.confirmPasswordInput.fill(confirm);
    await this.confirmPasswordInput.blur();
```

```
  }

  async submit() {
    await this.submitButton.click();
    await expect(this.successHeading).toBeVisible({ timeout: 45_000 });
  }
}
```

03_Automate_Code/tests/form-validation.spec.ts 162 lines

```
import { test, expect } from '../src/fixtures/test';
import { Messages } from '../src/data/messages';
import { buildRegistrant, uniqueId } from '../src/data/registrant';
import { env } from '../src/config/env';

test.describe('Registration form — required fields', () => {
  test('REG-VAL-01 empty submit (Thai company) shows every required-field error @smoke', async ({
registerPage }) => {
    await registerPage.submit();

    for (const message of [
      Messages.required.taxIdOrCompany,
      Messages.required.businessType,
      Messages.required.userAmount,
      Messages.required.email,
      Messages.required.firstName,
      Messages.required.lastName,
      Messages.required.phone,
      Messages.required.consent,
    ]) {
      await expect(registerPage.error(message)).toBeVisible();
    }
    await expect(registerPage.page).toHaveURL(/\/Register\/empeo/);
  });

  test('REG-VAL-02 empty submit (Others) asks for company name', async ({ registerPage }) => {
    await registerPage.chooseCompanyType('others');
    await registerPage.submit();

    await expect(registerPage.error(Messages.required.allRequired)).toBeVisible();
    await expect(registerPage.error(Messages.required.email)).toBeVisible();
  });

  test('REG-VAL-03 all fields valid except consent blocks submission', async ({ registerPage }) => {
    const submissions = registerPage.blockSubmissions();
    await registerPage.fill(buildRegistrant(), { acceptConsent: false });
    await registerPage.submit();

    await expect(registerPage.consentCheckbox).not.toBeChecked();
    await expect(registerPage.page).toHaveURL(/\/Register\/empeo/);
    expect(submissions).toHaveLength(0);
  });

  test('REG-VAL-05 consent error is shown when only consent is missing', async ({ registerPage }) => {
    test.fail(true, 'Known defect: submit is silently ignored, consent message is not displayed');
    registerPage.blockSubmissions();
    await registerPage.fill(buildRegistrant(), { acceptConsent: false });
    await registerPage.submit();

    await expect(registerPage.error(Messages.required.consent)).toBeVisible({ timeout: 3_000 });
  });

  test('REG-VAL-04 required errors clear once valid data is entered', async ({ registerPage }) => {
    await registerPage.submit();
    await expect(registerPage.error(Messages.required.email)).toBeVisible();

    await registerPage.fill(buildRegistrant());

    await expect(registerPage.visibleErrors()).toHaveCount(0);
  });
});
```

```
test.describe('Registration form - company type', () => {
  test('REG-CMP-01 switching company type toggles tax search and company name inputs', async ({
registerPage }) => {
    await expect(registerPage.taxSearchInput).toBeVisible();
    await expect(registerPage.companyNameInput).toBeHidden();

    await registerPage.chooseCompanyType('others');
    await expect(registerPage.taxSearchInput).toBeHidden();

    await registerPage.chooseCompanyType('thai');
    await expect(registerPage.companyNameInput).toBeHidden();
  });

  test('REG-CMP-02 tax search by company name returns DBD record and fills tax ID', async ({ registerPage
}) => {
    await registerPage.searchTaxId('โกไฟว์');
    await registerPage.selectFirstTaxResult('บริษัท โกไฟว์ จำกัด');

    await expect(registerPage.taxSearchInput).toHaveValue('0745561010054');
  });
});

test.describe('Registration form - email', () => {
  const malformedEmails = [
    { label: 'no at-sign', build: () => 'abc' },
    { label: 'no top-level domain', build: () => 'a@b' },
    { label: 'double at-sign', build: () => `user@@qa${uniqueId()}.com` },
    { label: 'leading space', build: () => ` spaced@qa${uniqueId()}.com` },
  ];

  for (const { label, build } of malformedEmails) {
    test(`REG-VAL-10 rejects malformed email (${label})`, async ({ registerPage }) => {
      await registerPage.fillEmail(build());
      await expect(registerPage.error(Messages.invalid.email)).toBeVisible();
    });
  }

  test('REG-VAL-11 accepts a well-formed, unused email', async ({ registerPage }) => {
    await registerPage.fillEmail(buildRegistrant().email);
    await expect(registerPage.error(Messages.invalid.email)).toBeHidden();
    await expect(registerPage.error(Messages.duplicate.email)).toBeHidden();
  });

  test('REG-VAL-12 rejects an email that already has an account', async ({ registerPage }) => {
    await registerPage.fillEmail(env.registeredEmail);
    await expect(registerPage.error(Messages.duplicate.email)).toBeVisible();
  });

  test('REG-VAL-13 rejects a new email on a company domain that is already registered', async ({
registerPage }) => {
    await registerPage.fillEmail(`qa.${uniqueId()}@${env.registeredDomain}`);
    await expect(registerPage.error(Messages.duplicate.domain)).toBeVisible();
  });

  test('REG-VAL-14 accepts a new email on a public mail provider domain', async ({ registerPage }) => {
    await registerPage.fillEmail(`qa.${uniqueId()}@gmail.com`);
    await expect(registerPage.visibleErrors()).toHaveCount(0);
  });
});

test.describe('Registration form - mobile number', () => {
  for (const phone of ['096769070', '09676907081', '1234567890', 'abcdef']) {
    test(`REG-VAL-20 rejects invalid mobile number "${phone}"`, async ({ registerPage }) => {
      await registerPage.fillPhone(phone);
      await expect(registerPage.error(Messages.invalid.phone)).toBeVisible();
    });
  }

  for (const phone of [env.phone, env.phone.replace(/^0/, '')]) {
    test(`REG-VAL-21 accepts valid Thai mobile number "${phone}"`, async ({ registerPage }) => {
      await registerPage.fillPhone(phone);
      await expect(registerPage.error(Messages.invalid.phone)).toBeHidden();
```

```
    });
  }
});

test.describe('Registration form — field length limits', () => {
  const overflow = 'a'.repeat(300);
  const limits = [
    { field: 'email', max: 150 },
    { field: 'firstName', max: 255 },
    { field: 'lastName', max: 255 },
    { field: 'phone', max: 15 },
  ] as const;

  for (const { field, max } of limits) {
    test(`REG-VAL-30 ${field} is capped at ${max} characters`, async ({ registerPage }) => {
      const input = {
        email: registerPage.emailInput,
        firstName: registerPage.firstNameInput,
        lastName: registerPage.lastNameInput,
        phone: registerPage.phoneInput,
      }[field];
      await input.fill(overflow);
      await expect(input).toHaveValue(overflow.slice(0, max));
    });
  }

  test('REG-VAL-31 company name (Others) is capped at 255 characters', async ({ registerPage }) => {
    await registerPage.chooseCompanyType('others');
    await registerPage.companyNameInput.fill(overflow);
    await expect(registerPage.companyNameInput).toHaveValue(overflow.slice(0, 255));
  });
});
```

03_Automate_Code/tests/promo-code.spec.ts 95 lines

```
import { test, expect } from '../src/fixtures/test';
import { Messages } from '../src/data/messages';
import { RegisterPage } from '../src/pages/RegisterPage';
import { env } from '../src/config/env';

test.describe('Promo code', () => {
  test('REG-PRM-01 valid promo code is accepted and shown as applied @smoke', async ({ registerPage }) => {
    const response = await registerPage.applyCoupon(env.promoCode);
    const body = await response.json();

    expect(response.ok()).toBeTruthy();
    expect(body.data).toMatchObject({ isValid: true, couponCode: env.promoCode });
    await expect(registerPage.appliedCoupon(env.promoCode)).toBeVisible();
    await expect(registerPage.error(Messages.invalid.coupon)).toBeHidden();
  });

  test('REG-PRM-02 promo code is case-insensitive', async ({ registerPage }) => {
    const code = env.promoCode.toLowerCase();
    const response = await registerPage.applyCoupon(code);

    expect((await response.json()).data).toMatchObject({ isValid: true, couponCode: env.promoCode });
    await expect(registerPage.appliedCoupon(code)).toBeVisible();
  });

  for (const code of ['INVALID99', 'FREE15DAYX']) {
    test(`REG-PRM-03 unknown promo code "${code}" is rejected`, async ({ registerPage }) => {
      const response = await registerPage.applyCoupon(code);

      expect(response.status()).toBe(400);
      await expect(registerPage.error(Messages.invalid.coupon)).toBeVisible();
      await expect(registerPage.couponInput).toBeVisible();
    });
  }

  test('REG-PRM-08 special-character promo code is rejected without applying', async ({ registerPage }) =>
{
    await registerPage.openCouponInput();
```

```
    await registerPage.couponInput.fill('!@#$%');
    await registerPage.couponApplyButton.click();

    await expect(registerPage.appliedCoupon('!@#$%')).toBeHidden();
    await expect(registerPage.couponInput).toBeVisible();
  });

  test('REG-PRM-04 applied promo code can be removed', async ({ registerPage }) => {
    await registerPage.applyCoupon(env.promoCode);
    await expect(registerPage.appliedCoupon(env.promoCode)).toBeVisible();

    await registerPage.removeCoupon();

    await expect(registerPage.appliedCoupon(env.promoCode)).toBeHidden();
    await expect(registerPage.couponInput).toBeVisible();
  });

  test('REG-PRM-05 promo code input is capped at 50 characters', async ({ registerPage }) => {
    await registerPage.openCouponInput();
    await registerPage.couponInput.fill('X'.repeat(80));
    await expect(registerPage.couponInput).toHaveValue('X'.repeat(50));
  });

  test('REG-PRM-06 exhausted / reused promo code is rejected (stubbed API)', async ({ registerPage, page })
=> {
    await page.route(RegisterPage.api.couponValidation, (route) =>
      route.fulfill({
        status: 400,
        contentType: 'application/json',
        body: JSON.stringify({
          status: { code: 4001, customMessage: '', description: 'Business error with  1 errors.' },
          data: { subCode: 'E1104', description: 'Coupon usage limit reached.', metaData: { couponCode: ''
}, customMessage: '' },
        }),
      }),
    );

    await registerPage.applyCoupon('USEDCODE01');

    await expect(registerPage.error(Messages.invalid.coupon)).toBeVisible();
    await expect(registerPage.appliedCoupon('USEDCODE01')).toBeHidden();
  });

  test('REG-PRM-07 coupon flagged isValid=false must not be applied (stubbed API)', async ({ registerPage,
page }) => {
    test.fail(true, 'Known defect: UI applies the coupon even when API returns isValid=false');
    await page.route(RegisterPage.api.couponValidation, (route) =>
      route.fulfill({
        status: 200,
        contentType: 'application/json',
        body: JSON.stringify({
          status: { code: 1000, customMessage: '', description: 'Success.' },
          data: { isValid: false, reason: 'Coupon usage limit reached', couponCode: 'USEDCODE02', isLock:
false, isReferral: false },
        }),
      }),
    );

    await registerPage.applyCoupon('USEDCODE02');

    await expect(registerPage.appliedCoupon('USEDCODE02')).toBeHidden({ timeout: 3_000 });
  });
});
```

03_Automate_Code/tests/registration-e2e.spec.ts 131 lines

```
import { test, expect } from '../src/fixtures/test';
import { Messages } from '../src/data/messages';
import { buildRegistrant, maskPhone, toE164 } from '../src/data/registrant';
import { OtpDialog } from '../src/pages/OtpDialog';
import { env } from '../src/config/env';
```

```
test.describe('Registration end-to-end on UAT @e2e', () => {
  test.describe.configure({ mode: 'default' });

  test('REG-E2E-01 register with promo code, verify OTP and set password @smoke', async ({ registerPage,
otpDialog, createPasswordPage }) => {
    test.setTimeout(240_000);
    const registrant = buildRegistrant({ promoCode: env.promoCode });

    await test.step('Fill the registration form', async () => {
      await registerPage.fill(registrant);
      await expect(registerPage.appliedCoupon(env.promoCode)).toBeVisible();
    });

    await test.step('Submit and receive OTP', async () => {
      const registration = await registerPage.submitAndWaitForRegistration();
      expect(registration.ok()).toBeTruthy();
      expect(registration.request().postDataJSON()).toMatchObject({
        isThaiCompany: false,
        companyName: registrant.company,
        email: registrant.email,
        firstName: registrant.firstName,
        lastName: registrant.lastName,
        phone: toE164(registrant.phone),
        couponCode: env.promoCode,
      });
      await otpDialog.expectOpen();
      await
expect(otpDialog.root.getByText(Messages.otp.sentTo(maskPhone(registrant.phone)))).toBeVisible();
    });

    await test.step('Verify OTP', async () => {
      const verification = await otpDialog.verify(env.otp);
      expect((await verification.json()).data).toMatchObject({ isValid: true });
      await createPasswordPage.waitForPage();
    });

    await test.step('Password rules gate the submit button', async () => {
      await createPasswordPage.fill('abc');
      await expect(createPasswordPage.submitButton).toBeDisabled();
      await createPasswordPage.fill(env.password.toLowerCase());
      await expect(createPasswordPage.ruleSatisfied('Aa')).toHaveCount(0);
      await expect(createPasswordPage.submitButton).toBeDisabled();
      await createPasswordPage.fill(env.password);
      for (const rule of ['Aa', '123', '8 ตัวอักษรขึ้นไป'] as const) {
        await expect(createPasswordPage.ruleSatisfied(rule)).toBeVisible();
      }
      await expect(createPasswordPage.submitButton).toBeEnabled();
    });

    await test.step('Set password and land on the success page', async () => {
      await createPasswordPage.submit();
      await expect(createPasswordPage.startButton).toBeVisible();
      await test.info().attach('registration-success', { body: await createPasswordPage.page.screenshot(),
contentType: 'image/png' });
    });
  });
});

test.describe('OTP verification @e2e', () => {
  test.beforeEach(async ({ registerPage, otpDialog }) => {
    await registerPage.fill(buildRegistrant());
    const registration = await registerPage.submitAndWaitForRegistration();
    expect(registration.ok()).toBeTruthy();
    await otpDialog.expectOpen();
  });

  test('REG-OTP-01 dialog shows masked phone, reference and countdown', async ({ otpDialog }) => {
    await expect(otpDialog.root.getByText(Messages.otp.title)).toBeVisible();
    await expect(otpDialog.root.getByText(Messages.otp.sentTo(maskPhone(env.phone)))).toBeVisible();
    await expect(otpDialog.reference).toBeVisible();
    await expect(otpDialog.countdown).toBeVisible();
    await expect(otpDialog.resendButton).toBeHidden();
```

```
  });

  test('REG-OTP-02 verify button stays disabled until all 6 digits are entered', async ({ otpDialog }) => {
    await expect(otpDialog.verifyButton).toBeDisabled();
    await otpDialog.enter('12345');
    await expect(otpDialog.verifyButton).toBeDisabled();
    await otpDialog.page.keyboard.type('6');
    await expect(otpDialog.verifyButton).toBeEnabled();
  });

  test('REG-OTP-03 non-numeric characters are not accepted', async ({ otpDialog }) => {
    await otpDialog.enter('abcdef');
    expect(await otpDialog.value()).toBe('');
    await expect(otpDialog.verifyButton).toBeDisabled();
  });

  test('REG-OTP-04 wrong OTP is rejected and the inputs are cleared', async ({ otpDialog, page }) => {
    const verification = await otpDialog.verify('000000');

    expect((await verification.json()).data?.isValid).not.toBe(true);
    await expect(page.getByText(Messages.otp.invalid)).toBeVisible();
    await expect.poll(() => otpDialog.value()).toBe('');
    await expect(page).toHaveURL(/\/Register\/empeo$/);
  });

  test('REG-OTP-05 resend issues a new reference and restarts the countdown', async ({ otpDialog }) => {
    test.setTimeout(120_000);
    const before = await otpDialog.referenceId();

    const resend = await otpDialog.resend();

    expect(resend.ok()).toBeTruthy();
    await expect.poll(() => otpDialog.referenceId()).not.toBe(before);
    await expect(otpDialog.countdown).toBeVisible();
  });

  test('REG-OTP-06 expired OTP shows an error and keeps the dialog open (stubbed API)', async ({ otpDialog,
page }) => {
    await page.route(OtpDialog.api.verifyOtp, (route) =>
      route.fulfill({
        status: 400,
        contentType: 'application/json',
        body: JSON.stringify({
          status: { code: 4001, customMessage: '', description: 'Business error with  1 errors.' },
          data: { subCode: 'E4002', description: 'OTP has expired.', metaData: {}, customMessage: '' },
        }),
      }),
    );

    await otpDialog.verify(env.otp);

    await expect(otpDialog.errorPopup).toBeVisible();
    await otpDialog.dismissErrorPopup();
    await otpDialog.expectOpen();
    await expect(page).toHaveURL(/\/Register\/empeo$/);
  });
});
```